

PLAIT Syntax

CMSC 323

Reminder: Slack and GitHub Classroom

If you haven't already, please accept Slack invite and send me your GitHub username.

PLAIT Syntax

PLAIT Scheme is derived from the scheme language. In addition, it includes the `define-type` and `type-case` forms, along with testing support.

Comments

Single line comments

`;;` This is a comment that continues to the end of the line.

Multiline comments

`#|` This is a block comment,
which starts with `"#|"`
and ends with a `"|#"`. Block comments
can be nested, which is why I can
name the start and end tokens in this
comment.

`|#`

Comments

`#;` comments out a block of code

Booleans

#t ; traditional name for true

#f ; ditto for false

Numbers

1

0.5

$1/2$; this is a literal fraction, not a division operation

$1+2i$; complex number

Strings

"apple"

"banana cream pie"

Symbols

'apple

'banana-cream-pie

'a->b

'#%\$^@*&?!'

Note: control sequences are still treated as control sequences. E.g., ``\ apple`

Built-in Functions on Atomic Data

not

+

-

* ; etc.

<

>

=

<=

>=

string-append

string=?

string-ref

eq? ; for symbols

equal? ; other types

empty?

Built-in Functions on Atomic Data

- ◆ Note, the following do not work in plait
 - number?
 - symbol?
 - string?
- ◆ Why do you think these are not included in plait?
 - Hint: They *are* included in racket

Basic Expression Forms

Syntax:

(expr expr...)

In English: The syntax for a function call is open pren, followed by an expression that must evaluate to a function, followed by one or more expressions that are evaluated to produce the operands, followed by a close pren.

Examples:

(not #t) ; => false

(+ 1 2) ; => 3

(< 2 1) ; => false

(= 1 1) ; => true

Basic Expression Forms

`(string-append "a" "b") ; => "ab"`

`(string-ref "apple" 0) ; => #\a`

Note that `#\` is the plait syntax prefix used to denote a literal character

Basic Expression Forms

`(eq? 'apple 'apple)` ; => true

`(eq? 'apple 'orange)` ; => false

`(eq? (string-append "app" "le")
"apple")` ; => false, probably

`(equal? (string-append "app" "le")
"apple")` ; => true

A Completely Arbitrary Aside

- ◆ The compiler question from yesterday...
- ◆ I found out what plait stands for!
 - plai: **P**rogramming **L**anguages **A**pplication and **I**nterpretation
 - ◆ Like racket, this is a dynamically typed language
 - plait: PLAI typed
 - ◆ Which, as we've discussed, is statically typed

Another aside, but quite useful

- ◆ Contracts: Basically function signatures that state input and return types.
- ◆ Plait enforces these, and you need to use them in your projects
- ◆ An example:

Basic Expression Forms

(string=? "apple" "apple"); => true

(empty? empty) ; => true

Conditionals

Syntax:

```
(cond  
  [test-expr expr]...)
```

```
(cond  
  [test-expr expr]...  
  [else expr])
```

A sequence of clauses between square brackets. In each clause, the first *test-expr* is a test expression. If it produces `#t`, then the clause's remaining *expr* is evaluated, this provides the answer for the entire `cond` expression; the rest of the clauses are ignored. If the test *test-expr* produces `#f`, then the clause's *expr* is ignored, and evaluation continues with the next clause.

Conditionals

Examples:

```
(cond  
  [(< 2 1) 17]  
  [(> 2 1) 18])
```

; => 18

```
(cond  
  [#t 8]  
  [#f 4])
```

; 8

Conditionals

Examples:

```
(cond ; any number of cond-lines allowed
  [(< 3 1) 0] ;
  [(< 3 2) 1] ;
  [(< 3 3) 2] ;
  [(< 3 4) 3]) ; => 3
```

```
(cond ; else allowed as last case
  [(eq? 'a 'b) 0] ;
  [(eq? 'a 'c) 1] ;
  [else 2]) ; => 2
```

If

(**if** *test-expr* *expr* *expr*)

The *test-expr* is always evaluated first. If it produces a **#t** value, then the first *expr* is evaluated for the result of the whole **if** expression, otherwise the second *expr* is evaluated for the result.

and and or

Syntax:

(and *expr* ...)

(or *expr* ...)

Examples:

(and #t #t) ; => true

(and #t #f) ; => false

(and (< 2 1) #t) ; => false

and and or

What about this?:

(and (< 2 1) (+ 'a 'b)) :=> false (second expression not evaluated)

and and or

Examples:

(or #f #t) ; true

(or #f #f) ; false

Built-in Compound Data

Functions to create compound data items

list

cons

Built-in Compound Data

Example:

`(cons a d)`

The above returns a newly allocated pair element whose first element is `a` and second element is `d`. `d` is a list of elements with the same type as `a`.

Built-in Compound Data

Examples:

`(cons 1 empty)` ; => `(list 1)`

`(cons 1 (cons 2 empty))` ; => `(list 1 2)`

Built-in Compound Data

list

(list 1 2 3) ; => (list 1 2 3)

Built-in Compound Data

append

(append (list 1 2) empty) ; => (list 1 2)

(append (list 1 2)
 (list 3 4)) ; => (list 1 2 3 4)

Built-in Compound Data

first, rest, and list-ref

```
(first (list 1 2 3))    ;=> 1
```

```
(rest (list 1 2 3))     ;=> (list 2 3)
```

```
(first (rest (list 1 2))) ;=> 2
```

```
(list-ref '(1 2 3) 2)   ;=> 3
```

Built-in Compound Data

'(...) creates a list, and it distributes over elements

ditto for '{}'

```
'(1 2 3)           ;=> (list 1 2 3)
'(a b)             ;=> (list 'a 'b)
'((1 2) (3 4))     ;=> (list (list 1 2) (list 3 4))
'10                ;=> 10
'{{{c}}}}          ;=> (list (list (list 'c)))
```

Definitions

Defining Constants

Syntax:

`(define id expr)`

Binds *id* to *expr* (or equivalently binds *expr* to *id*)

Example:

`(define PI 3.14159)`

`(* PI 10) ; => 31.4159`

Definitions

Defining Functions

Syntax:

```
(define (id field-id ...) expr)
```

id is the function name *field-ids* the function parameters and *expr* is the body of the function.

Example:

```
(define (circle-area r)
```

```
  (* (* PI r) r))
```

; function definition

```
(circle-area 10)
```

; function call that returns 314.159

Let's have some fun(?)

Define the functions `is-even` and `is-odd`

Syntax:

```
(define (id field-id ...) expr)
```

id is the function name *field-ids* the function parameters and *expr* is the body of the function.

Defining Data Types

Syntax:

```
(define-type type-id variant ...)
```

```
variant = (variant-id (field-id contract-expr) ...)
```

define-type defines the datatype *type-id*. A constructor *variant-id* is defined for each variant. Each constructor takes an argument for each field of its variant.

The value of each field is checked by its associated *contract-expr*.

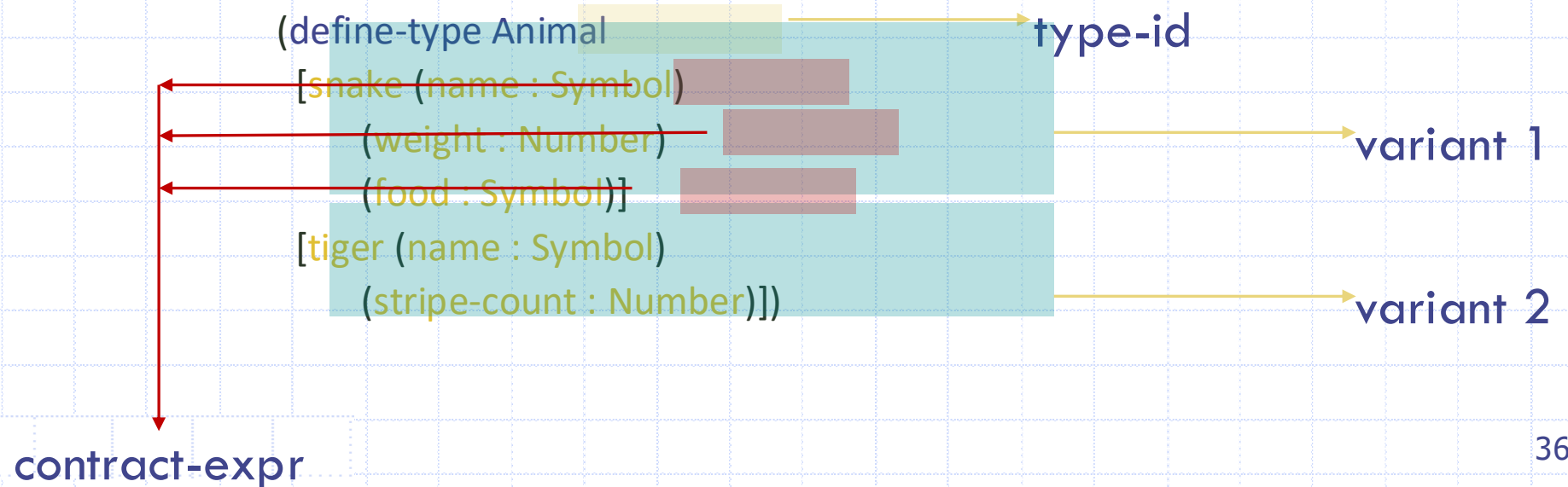
Lot of stuff here that we haven't defined (yet)

define-type

Example:

```
(define-type type-id variant ...)
```

```
variant = (variant-id (field-id : contract-expr) ...)
```



define-type

A type can have any number of variants.

```
(define-type Shape  
  [square (side : Number)]  
  [circle (radius : Number)]  
  [triangle (height : Number)  
            (width : Number)])
```

```
(triangle? (triangle 10 12))      ; => true
```

define-type

A constructor *variant-id* is defined for each variant. Each constructor takes an argument for each field of its variant.

The value of each field is checked by its associated *contract-expr*.

```
(define-type Animal
  [snake (name : Symbol)
        (weight : Number)
        (food : Symbol)]
  [tiger (name : Symbol)
        (stripe-count : Number)])
```

variant-id

arguments for the snake constructor/variant

Using Datatypes

Creating instances of a data type

```
(snake 'Slimey 10 'rats) ; => (snake 'Slimey 10 'rats)  
(tiger 'Tony 12)        ; => (tiger 'Tony 12)
```

Using Datatypes

define-type defines a predicate *variant-id?* that returns **true** for instances of the *variant-id*, and **false** for any other *variant-type*.

```
(snake? (snake 'Slimey 10 'rats)) ;=> true
```

```
(tiger? (tiger 'Tony 12))           ;=> true
```

```
(triangle? 10)                       ;=> error
```

Using Datatypes

define-type **defines** a predicate *variant-id?* that returns **true** for instances of the *variant-id*, and **false** for any other *variant-type*. (You get this for free.)

```
(snake? (snake 'Slimey 10 'rats)) ; => true
```

```
(tiger? (tiger 'Tony 12)) ; => true
```

```
(triangle? 10) ; => error
```

What is a predicate?

Using Datatypes

For each variant, *define-type* defines a predicate *variant-id?* that returns **true** when applied to a value of the same variant and false for any other value.

```
(snake? (snake 'Slimey 10 'rats)) ; => true  
(tiger? (snake 'Slimey 10 'rats)) ; => false
```

Using Datatypes

For each field of each variant, *define-type* defines an **accessor** *variant-id-field-id* that returns the value of the field

(You get this for free too. Lots of free stuff)

```
(snake-name (snake 'Slimey 10 'rats)) ; => 'Slimey  
(tiger-name (tiger 'Tony 12)) ; => 'Tony
```

Datatype Case Dispatch

```
(type-case datatype-id expr  
  branch ...)  
branch = (variant-id (field-id ...) result-expr ...)
```

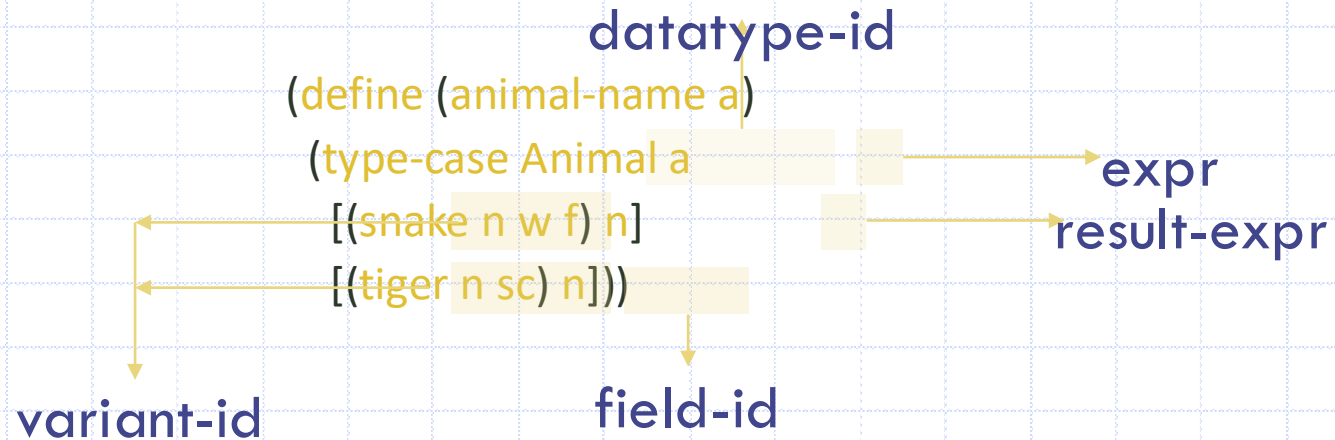
Branches on the datatype instance produced by *expr*, which must be an instance of *datatype-id* (previously defined with *define-type*). Each branch extracts the values of the fields and binds them to *field-id*.

Use **else** for a catch-all branch. An else branch must be the last branch in the sequence of branches.

Datatype Case Dispatch

(type-case *datatype-id* *expr*
 branch ...)

branch = (*variant-id* (*field-id* ...) *result-expr* ...)



Local Binding Forms

`(local [definition ...] expr)`

The `local` form accommodates multiple definitions that are visible only among the definitions and the body `expr`.

Example:

```
(local [(define x 10)]  
  (+ x x))      ; => 20
```

```
#;x              ; => error: unbound identifier
```

Recall that `#;` is another comment. Comments out the following single complete balanced expression

Local Binding Forms

Example:

```
(define (to-the-fourth x)
  (local [(define (squared n)
                (* n n))]
    (* (squared x) (squared x))))
(to-the-fourth 10)
```

Local Binding Forms

let

Syntax:

`(let ([id expr]...) expr)`

Example:

```
(let ([x 10]
      [y 11])
  (+ x y))
```

Note: This is not used in the Findler course, but we use it.

Reasonable question: What is the difference between the two?

local: every definition defined inside the local block is visible to all other definitions, regardless of the order in which they are written.

let: The expressions on the right-hand side can only see variables from the outer scope. They cannot see the variables being defined in the let block, nor can they see sibling variables defined in the same let block.

Local Binding Forms

◆ You can't do this with "let" (why not?)

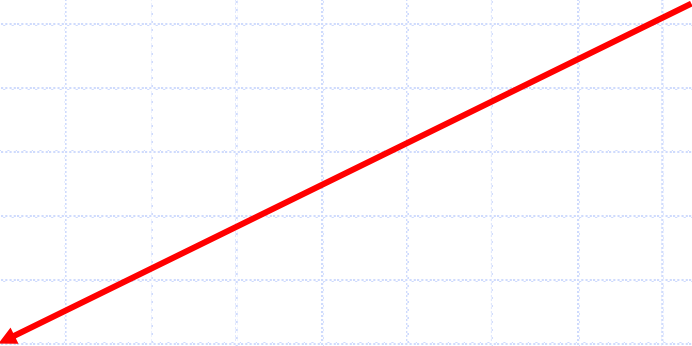
```
(local [(define (even? n)
  (if (= n 0)
      #t
      (odd? (- n 1))))
  (define (odd? n)
    (if (= n 0)
        #f
        (even? (- n 1))))]
  (even? 4))
```

Anonymous Functions

Syntax:

`(lambda (id ...) expr)`

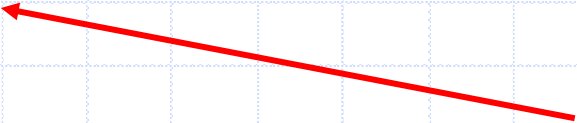
This is a function that *returns*
the defined function



Examples:

`(lambda (x) (+ x 1)) ; => (lambda (a1) ...)`

`((lambda (x) (+ x 1)) 10) ; => 11`



This one *calls* the defined function

Anonymous Functions

Syntax:

`(lambda (id ...) expr)`

Examples:

`(lambda (x) (+ x 1)) ; => (lambda (a1) ...)`

`((lambda (x) (+ x 1)) 10) ; => 11`

Quick task: use a lambda to make a function that uses two input parameters.

Anonymous Functions

Example 1:

Associate a function with a name:

```
(define add-one  
  (lambda (x) (+ x 1)))
```

Invoke the function:

```
(add-one 10)  ;=> 11
```

Anonymous Functions

Example 2:

:=> returns a function that adds n to a number

```
(define (make-adder n)  
  (lambda (m)  
    (+ m n)))
```

:=> returns a function

```
(define add-ten (make-adder 10))  
(add-ten 5)
```

:=> 15

List Iteration

(map *proc lst* ...) -> *list*?

proc : procedure?

lst : *list*?

map applies *proc* to the elements of the *lst* from the first element to the last.

Very handy!

List Iteration

There is no map3, etc. But you can accomplish the same functionality using these two functions. You can play with that to figure out how.

Example 1:

```
(map (lambda (x) (* x x))  
      '(1 2 3))  
      ;=> (list 1 4 9)
```

Example 2:

```
(map2 (lambda (x y) (* x y))  
       '(1 2 3)  
       '(2 3 4))  
      ;=> (list 2 6 12)
```

filter

`(filter proc lst)`

Returns a list containing (in order) the items of a given list for which a given function returns true.

`(filter (lambda (x)`

`(> x 2))`

`'(1 3 5)) ; '(3 5)`

foldl

Note the order of these



`(foldl proc init lst)`

Applies *proc* to *init* and each element in *lst*, each time obtaining a new accumulated value.

`(foldr (lambda (n r)`

`(+ n r))`

`0`

`'(1 2 3))` ; *r* is initially 0, the function returns 6

Side-effects

set!

(set! *id expr*)

Mutates *id* to have the value of *expr*.

(define count 0)

(set! count (+ count 1)) ; =>

count ; => 1

Side-effects

begin

Syntax:

(begin *expr* ...)

Evaluates the *exprs* in sequence, producing the result of the last *expr*.

Side-effects

begin

Example 1:

```
(define count 0)
(begin
  (set! count (+ count 1))
  count)
```

Side-effects

box is like a vector with a single slot. Boxes are used primarily to allow mutation. For example, the value of a field in a variant instance cannot be modified, but the field's value can be a box, and then the box's content can be modified.

box function creates a box with an initial value for its slot

unbox accesses the current value in a box slot

set-box! Changes the value

Side-effects

Example:

```
(define-type Shape  
  [square (side : (Boxof Number))]  
  [rectangle (length : Number)  
             (width : Number)])
```

```
(define shape (square (box 5)))  
(unbox (square-side shape))           ;=> 5
```

```
(define side (square-side shape))  
(set-box! side 6)  
(unbox (square-side shape))           ;=> 6
```

Testing

test

Example 1:

(test (+ 5 5) 10)

Result:

good (+ 5 5) at line 207

expected: 10

given: 10

Example 2:

(test (+ 5 4) 10)

Result:

bad (+ 5 4) at line 208

expected: 10

given: 9

Testing

`(test/expr result-expr error-message)`

This test succeeds if the expression evaluates to a call to error.

```
(define (area1 radius)
  (if (> radius 0)
      (* (* radius radius) 22/7)
      (error 'area1 "radius should be greater than 0")))
```

```
(test (area1 7) 154)
(test/expr (area1 -1) "radius should be greater than 0")
```

Testing

Use the following to show testing output **only** when there are failures:

```
(print-only-errors #t)
```